

Seller Dashboard for the watsonx IBMer Challenge

Automating a four-tool outbound sales workflow behind three buttons, with IBM watsonx.ai (Granite) doing the last-mile personalization.

Status: Draft

Owner: Sydney Chin

Date: July 16, 2026

Submission needs: 3-minute demo video · 599-word problem statement · 500-word technical statement

Judging weight (per prompt): Productivity impact and effective use of watsonx rank highest — this PRD is built to maximize both.

The one thing to fix before anything else: this repo is branded "WatsonX Clone" but currently makes zero calls to IBM watsonx — its live AI feature calls the Anthropic Claude API. For a challenge named after watsonx, that is the single biggest risk to this entry. Swapping to watsonx.ai (Granite) is Priority 0. See Section 5.

0 The One Thing to Fix Before Anything Else

This repo is branded "**WatsonX Clone**," but today it contains **zero calls to IBM watsonx**. The one live-AI feature (Bobby's personalized emails, plus the tiering "Play + Angle" and call-plan coaching note) calls the **Anthropic Claude API** (`llm_advisor.py`).

Risk: for a challenge literally named after watsonx, submitting this as-is is the single biggest risk to the entry — a judge who opens the code finds `api.anthropic.com` , not IBM. Section 5 makes swapping this to watsonx.ai (Granite) a **P0**, not a nice-to-have.

1 Problem Statement (source for the 599-word field)

A B2B infrastructure seller's outbound motion is stitched together from four disconnected systems: a Salesforce-based territory tool (ISC) for which accounts are theirs, an internal install-base portal for what those accounts already own, ZoomInfo for company/contact enrichment, and Salesloft for actually running outreach cadences. Building one week of a real outbound plan means: pulling every account in a territory one Coverage ID at a time, cross-referencing install data by hand, manually judging which accounts are worth a call and why, building a call calendar around the rest of a rep's schedule, finding the right contacts at each account, and writing a personalized first-touch email for every one of them. None of these systems talk to each other, so a seller either burns hours a week doing this manually or skips it and defaults to whatever accounts are already top-of-mind — which means under-covered territory and generic, low-response outreach.

The Seller Dashboard automates that entire chain end-to-end, behind three buttons: **Get My Accounts** (territory pull → install-base join → segmentation), **Outbound Strategy** (tiering → a dial calendar through year-end), and **Fill Contacts to Salesloft** (contact readiness → cadence load). The last mile — writing a genuinely personalized outreach email per contact, referencing their title, company, and where they sit in the cadence — is where a foundation model earns its place: **Bobby, the AI Emailer**, powered by watsonx.ai's Granite models, drafts a tailored email for every person on an email step of a cadence in seconds, instead of a seller spending 5–10 minutes per contact hand-writing (or skipping) that personalization. The same Granite layer also turns a seller's deterministic account score into a one-line, seller-voiced reason to call — "why this account, why now" — rather than a bare number.

Applied to a real territory (the app's working target is ~200 high-quality staged contacts/day through year-end), this collapses a multi-hour, multi-tool weekly ritual into a ~10-minute run a seller can do before their first call block. The productivity story isn't "AI writes emails" in isolation — it's that watsonx sits at the end of a fully automated data pipeline, so the model always has real, current, account-specific context (install footprint, spend trend, competitor presence, cadence step) to write from, which is what makes its output usable without heavy editing rather than a generic mail-merge.

~330 words as written — trim to fit 599 alongside whatever intro/close framing the final submission needs; this is meant as raw material, not a final draft.

2 Solution Overview

A single local web app (Flask, `run_pipeline.py`) that runs a 5-step pipeline — ISC pull → install-base join → segmentation → tiering → call planning — then hands off to two more steps — contact readiness → Salesloft load — and finally to **Bobby**, the watsonx-powered emailer. Three buttons on one screen; each streams its own live log; results render as sortable tables, a dial calendar, and a drafted-email review page with a **Send All** action.

For the challenge, the app runs against **mocked** IBM/ZoomInfo/Salesloft data (`fake_data.py`) so it's demoable anywhere with no VPN, no logins, and no real customer data — but the watsonx call itself is **live**, against a real watsonx.ai project, on every run.

3 Target User & Personas

- **Primary — IBM infrastructure seller (BTSS/TSS rep).** Owns a territory of Coverage IDs, is judged on pipeline coverage and dial activity, currently does this workflow by hand or skips it.
- **Secondary — judge persona for this challenge.** IBM technical/sales leadership evaluating whether watsonx can be embedded in a real internal workflow, not just a chatbot.

4 Goals & Success Metrics

GOAL	METRIC	TARGET FOR THE DEMO
Collapse the manual research → outreach cycle	Wall-clock time from "start" to "contacts staged + emails drafted"	Under ~10 minutes for a territory, on camera
Make watsonx output usable, not generic	% of drafted emails referencing account-specific detail (install, title, cadence step)	100% of drafted emails cite at least one real signal
Scale personalization	Emails drafted per run without per-contact manual writing	One click → every person on an email step, batched
Prove it's really watsonx	Judge can see the actual watsonx.ai call (model id, project, request)	Shown on screen, not asserted in narration

5 Technical Plan — P0: Real watsonx.ai Integration

Today

`llm_advisor.py` calls `api.anthropic.com` with a Claude model (`claude-opus-4-8` by default) for three things, all fail-soft (the app works without a key, degrading to deterministic templates):

1. `advise_accounts()` — Account Tiering's Primary Play + Sales Angle.
2. `advise_plan_summary()` — Call Planning's coaching note.
3. Bobby's per-contact email draft (`Bobby_AI_Emailer/emailer.py`).

Target

Replace the Anthropic client with the **IBM watsonx.ai** REST API (or `ibm-watsonx-ai` SDK), same fail-soft contract, same call sites:

- New `watsonx_advisor.py` (or refactor `llm_advisor.py` in place) using `POST /ml/v1/text/generation` (or the `/text/chat` endpoint) against a provisioned watsonx.ai project, with a **Granite** instruct model (e.g. `ibm/granite-3-8b-instruct` or the latest available Granite chat model) as the default.
- New env vars, same pattern as the existing `.env.example` : `WATSONX_API_KEY` , `WATSONX_PROJECT_ID` , `WATSONX_URL` (region endpoint), `WATSONX_MODEL_ID` . `available()` becomes "these are all set," same as today's Anthropic key check.
- Keep the three call sites' system/user prompts and JSON-extraction contract unchanged where possible — this is a provider swap, not a redesign, which keeps risk low this close to a submission deadline.
- Keep the deterministic fallback path exactly as-is (tier numbers and calendar dates never depend on the model; only the narrative layer does) — this is already true today and is worth keeping in the technical statement as a reliability point.
- Rename user-facing "Claude" mentions (README, UI copy "Claude if ANTHROPIC_API_KEY is set") to watsonx/Granite so nothing in the demo or docs contradicts the pitch.

Stretch (only if P0 lands with time to spare): a second, more visible watsonx surface — e.g. routing the three Outbound buttons through a small **watsonx Orchestrate** flow instead of direct Flask routes, so the demo can show an agentic orchestration layer, not just a single completion call. Do not start this until P0 is done and demoed end-to-end at least once.

Out of scope for this challenge: real ISC/ZoomInfo/Salesloft connectivity (stays mocked — that's a deliberate, disclosed simplification, not a gap to hide), the Meetings/Pipeline tabs (already removed from this clone).

6 Why This Wins (rubric alignment)

LIKELY CRITERION	HOW THIS ENTRY ANSWERS IT
Productivity impact	Concrete before/after: multi-hour, multi-tool manual weekly ritual → one ~10-minute run. Not hypothetical — the pipeline actually runs.
Effective use of watsonx	Granite in the critical path of the "why did the AI help" story (personalized email + call rationale), fed by real pipeline context, not a demo chatbot bolted on the side.
Technical execution	A working 5+2-step orchestrated pipeline with a real file-handoff contract, live-streamed logs, and a fail-soft AI layer — not a slide deck.
Innovation / originality	AI positioned as the last mile of an automation chain, not the whole product — the differentiator is that watsonx always has fresh, structured, account-specific context to write from.
Demo quality / storytelling	See Section 7 — a tight, screen-recorded, narrated walkthrough hitting the pain, the run, and the watsonx moment inside 3 minutes.

7 Demo Video Plan (target: 3:00)

Screen-recorded walkthrough + voiceover. Goal: a judge who has never seen this tool understands the pain, watches the whole pipeline run once, and sees the actual watsonx call happen — all inside 180 seconds.

Scene 1 — The Pain (0:00–0:20)

On screen: a quick, fast-cut montage or split-screen — ISC account list, install-base spreadsheet, ZoomInfo tab, Salesloft — to visualize "four disconnected systems."

VO: "Every week, a seller manually stitches together four systems just to know who to call and what to say. That's hours of research before a single dial."

Scene 2 — The One-Liner (0:20–0:30)

On screen: app title card / login gate.

VO: "This is the Seller Dashboard — three buttons that run that whole chain end to end, with watsonx doing the last, hardest mile: personalization."

Scene 3 — Get My Accounts (0:30–0:55)

Screen-record: click **Get My Accounts**, show the live log streaming, land on the segmented-accounts result table.

VO: "One click pulls the seller's territory, joins it against install data, and segments it — accounts they didn't have to touch a single tool for."

On-screen text overlay: "~280 accounts, one click."

Scene 4 — Outbound Strategy (0:55–1:20)

Click **Outbound Strategy** → tiering table appears (call out a Tier 1 row's Play + Angle) → dial calendar.

VO: "Outbound Strategy scores and tiers every account, then lays out exactly which day to call which account through year-end."

On-screen callout: highlight one account's "Sales Angle" text with a circle/arrow: **"That reasoning — that's watsonx, not a template."**

Scene 5 — The watsonx Moment (1:20–2:05) — the most important 45 seconds

Click **Fill Contacts to Salesloft**, then **Run Bobby**. Show the Bobby review page populating with drafted emails in near-real-time. Cut to (or overlay) a clean visual of the actual watsonx.ai request: model id (**ibm/granite-3-8b-instruct** or whichever is final), the project, maybe a terminal/network-tab glimpse of the call and response — make it undeniable that this is a live IBM watsonx call, not a screenshot.

VO: "This is the moment that used to take a seller 5–10 minutes per contact. Bobby reads the cadence, the contact's role, their company, and drafts a specific, sendable email — powered by watsonx.ai's Granite models, grounded in the real account context the pipeline already built."

On-screen text: "Every email cites something real — install, title, cadence step. Not a mail-merge."

Scene 6 — Payoff / Scale (2:05–2:35)

Scroll through 3–4 drafted emails for different people to show variety, then click **Send All**.

VO: "One click, every person on an email step — not one email, all of them."

On-screen stat card: "Manual: ~5–10 min/email. Bobby: seconds, batched."

Scene 7 — Close (2:35–3:00)

Cut back to the three-button dashboard, all cards showing "done."

VO: "Four tools, hours of manual work, collapsed into one run — with watsonx doing the part that actually needed judgment: making every message specific."

End card: project name, one-line tagline, "Built with IBM watsonx.ai (Granite)."

Production Notes

- Record the P0 watsonx swap **before** filming — the demo must show a real watsonx call, not Claude with different labels.
- Keep every on-screen wait (pipeline steps, Bobby drafting) sped up in the edit; don't burn video time on real loading spinners.
- Have 2–3 backup takes of Scene 5 — if anything in the demo has to be bulletproof, it's the AI moment.
- Captions/on-screen text throughout — judges may watch muted.

8 Deliverables Checklist

- ☐ watsonx.ai swap complete (`WATSONX_*` env vars, Granite model live in Bobby + tiering + call-plan enrichment), verified end-to-end.
- ☐ README/UI copy scrubbed of "Claude"/"Anthropic" references.
- ☐ Full pipeline run rehearsed at least 3x for demo reliability.
- ☐ 3:00 demo video recorded, edited, captioned.
- ☐ 599-word problem statement (draft in Section 1).
- ☐ 500-word technical statement (draft in Section 9).
- ☐ Submitted via ibmer.watsonx-challenge.ibm.com before deadline.

9 Technical Statement Outline (source for the 500-word field)

Aim for this shape when drafting the final 500 words:

- What it is** (~60 words) — 5+2-step orchestrated pipeline, Flask, file-handoff contract between steps, watsonx.ai in the personalization layer.
- Where watsonx sits** (~150 words) — three call sites (tiering rationale, call-plan coaching note, Bobby's per-contact emails), Granite model, prompts grounded in structured account context assembled by the earlier pipeline steps, JSON-contract responses, fail-soft design (deterministic fallback if the model is unavailable — reliability without sacrificing the AI feature).
- Why the architecture matters** (~150 words) — separating deterministic scoring (tier numbers, calendar dates — reproducible, auditable) from AI narrative enrichment (Play/Angle, coaching notes, emails) means the model never controls a business-critical number, only the human-facing judgment layer — a pattern that generalizes to other IBM seller-tooling use cases.
- What's mocked and why** (~80 words) — ISC/ZoomInfo/Salesloft are deterministically faked for a portable, no-VPN, no-credential demo; the watsonx call is the one live, real integration in the app.
- What's next** (~60 words) — stretch: watsonx Orchestrate for the button-triggered flows, real ISC/ZoomInfo/Salesloft connectors behind the same interfaces already defined by the mocked modules.

10 Risks & Mitigations

RISK	MITIGATION
No watsonx.ai project/credentials provisioned yet	Provision first, before any other work — this blocks everything else in Section 5.
Granite output quality/latency worse than Claude in early testing	Budget time to tune prompts per call site; keep the existing fail-soft fallback as a safety net during rehearsal.
Demo-day network/API flake during recording	Record Scene 5 multiple times; consider a pre-captured "known good" run as backup B-roll, clearly still real watsonx output.
Judging rubric turns out to weight something else heavily (unknown — portal is auth-gated)	Re-validate this PRD against the actual rubric on the portal once accessible; Section 6 is written to hedge across the most common hackathon criteria (impact, technical depth, innovation, use of the sponsor's tech, demo quality).
"WatsonX Clone" repo name reads as already using watsonx before it does	Land the P0 swap before anyone (esp. judges) can view the repo, or don't share the repo link until it's done.

11 Open Questions (verify against the portal)

- Exact judging rubric / weighting.
- Team size / solo eligibility.
- Whether a public repo link is required/allowed, or the video + statements are the entire submission.
- Which watsonx.ai region/plan is available for IBMer challenge participants (affects `WATSONX_URL` and model availability).